

## Domain 4 — Section Review

Prompt Engineering &amp; Structured Output

11/14

79% · Needs review · 3 missed

## Q1 ✓ CORRECT

Your code review agent produces too many false positives on style issues. You add the instruction: "Only report high-confidence findings." False-positive rate stays unchanged. Why does this fail, and what is the correct fix?

- A Fails because the model has no calibrated confidence metric. Fix: reduce temperature to 0 to eliminate low-confidence outputs.
- B Fails because confidence-based instructions are vague and don't change what gets flagged. Fix: define explicit categorical criteria — which issue types to report and which to skip.**
- C Fails because the instruction is in the wrong position. Fix: move it to the beginning of the system prompt.
- D Fails because style and security reviews require separate prompt passes.

**Why B is correct:**

**Specific categorical criteria obliterate vague confidence-based instructions.** "High-confidence" has no operational definition. The correct structure: What to flag (bugs, security vulnerabilities). What to skip (minor style preferences, local patterns). Example: "Flag comments only when claimed behaviour contradicts actual code behaviour" — not "check that comments are accurate."

## Q2 ✓ CORRECT



One category — 'potential performance issues' — has a very high false-positive rate, causing developers to dismiss all review output including accurate security findings. The counterintuitive but correct short-term fix is:

- A Lower the confidence threshold specifically for the performance category to reduce noise in that category.
- B Add more detailed prompt instructions specifically for the performance category.
- C Temporarily disable the performance category entirely while improving its prompt, restoring developer trust in the remaining categories.**
- D Run the performance category as a separate pass so its noise doesn't appear alongside security findings.

**Why C is correct:**

**One noisy category contaminates trust in all categories.** Counterintuitive fix: pull the noisy category from output entirely — restore developer trust in what remains — then fix and re-enable it. Leaving it with more instructions (A, B) keeps noise flowing. Running separately (D) doesn't fix the category's prompt and still trains developers to ignore output.

## Q3 ✓ CORRECT

Your pipeline frequently returns `null` for `methodology_section` even when the methodology is clearly described inline in document prose (not in a dedicated section header). Most effective fix:

- A Make `methodology_section` a required field — the model won't skip it.
- B Add more `max_tokens` headroom so the model has space to extract all fields.

- C** Add 2–4 few-shot examples showing correct extraction from inline methodology descriptions versus dedicated section headers.



- D** Use tool\_choice: "any" to force a tool call instead of conversational text.

**Why C is correct:**

**Few-shot examples for structural variety.** When extraction fails because document structure varies, examples showing both patterns dramatically reduce empty field extraction — the model learns valid extraction shapes across formats. Making the field required (A) is a fabrication factory: the model will invent a methodology to satisfy the schema. max\_tokens (B) is irrelevant to extraction logic.

**Q4** ✓ **CORRECT**

Using `tool_use` with a strict JSON schema, your validation still catches errors: invoice line items don't sum to the stated total, and dates appear in the wrong fields. Why does this happen despite the strict schema?

- A** The JSON schema isn't being applied to nested array fields correctly.
- B** `tool_use` with strict schemas eliminates syntax errors but does not prevent semantic errors like arithmetic inconsistency or wrong-field placement.
- C** The schema needs tool\_choice: "any" to enforce full validation end-to-end.
- D** strict: true must be explicitly set — JSON schemas are advisory by default.

**Why B is correct:**

**tool\_use guarantees syntactic validity only.** Schema validates structure and types — it cannot enforce arithmetic relationships (line items sum to total), semantic field placement (right value in right field), or

logical consistency. These are semantic errors requiring application-layer validation. tool\_use eliminates JSON parse failures; it does not encode business logic.

**Q5** ✓ **CORRECT**

You're building a schema to extract contractor information from invoices. About 40% of invoices include a `tax_id` field; 60% do not. You define `tax_id` as a required string. What happens on the 60% that lack it?

- A The model returns an error indicating the field is absent.
- B The model skips the field and returns the schema without it.
- C The model fabricates a value to satisfy the required field constraint.**
- D The extraction fails and returns null for the entire document.

**Why C is correct:**

**Required fields = fabrication factories.** A required field the source doesn't contain pressures the model to invent something. Fix: make `tax_id` optional/nullable. The model can then honestly return null. Anti-fabrication toolkit: nullable fields for conditionally-present information, 'unclear' enum values for ambiguous cases, 'other' + detail string for extensible categories.

**Q6** ✓ **CORRECT**

Your team runs two workflows: (1) a blocking pre-merge code review developers wait on, and (2) a nightly technical debt audit reviewed next morning. Your manager proposes switching both to the Message Batches API for 50% cost savings. How should you evaluate this?

- A Switch both — the 50% savings applies to all workloads and batches often complete in

minutes.



- B** Keep both on real-time to avoid batch result ordering complexity.
- C** Use Message Batches API for the nightly audit only; keep real-time for pre-merge checks.
- D** Switch both with a 10-minute polling fallback to real-time if batches take too long.

**Why C is correct:**

**Match API to latency requirements.** Message Batches API: 50% cost savings, up to 24-hour processing window, no guaranteed latency SLA — unsuitable for blocking workflows. Pre-merge checks are blocking → real-time. Nightly audits are latency-tolerant → batch. Polling (D) doesn't help if a batch takes 24 hours — there's no guaranteed lower bound on completion time.

**Q7** ✓ **CORRECT**

A PR spanning 14 files produces inconsistent review depth — detailed on some files, superficial on others, and contradictory findings (flagging an identical pattern as problematic in one file, acceptable in another). Root cause and fix:

- A** Root cause: context window too small. Fix: switch to a higher-tier model with a larger context window.
- B** Root cause: attention dilution from processing many files simultaneously. Fix: split into per-file local analysis passes plus a separate cross-file integration pass.
- C** Root cause: inconsistent file formatting. Fix: auto-format all files before the review runs.
- D** Root cause: overly broad scope. Fix: restrict the prompt to security issues only.

**Why B is correct:**

**Attention dilution in multi-file single-pass review.** Larger context windows don't solve attention quality — they just give more to dilute across. The architectural fix: per-file passes for local issues (consistent depth per file), then a separate cross-file integration pass for data flow and interface consistency. Running three independent passes and requiring 2-of-3 consensus would suppress real intermittent bugs.

**Q8** ✓ **CORRECT**

Your automated reviews identify valid issues but developers report the feedback isn't actionable. Findings say things like 'complex ticket allocation logic' or 'potential null pointer' without specifying what to change. When you add detailed instructions like 'always include specific fix suggestions,' the model still produces inconsistent output — sometimes detailed, sometimes vague. What prompting technique would most reliably produce consistently actionable feedback?

- A** Add more detailed step-by-step instructions covering every possible finding type and how to suggest fixes for each.
- B** Reduce temperature to 0 to get deterministic, consistently detailed output.
- C** Add 2–4 few-shot examples demonstrating complete, actionable finding entries — each showing location, issue description, root cause, and a specific suggested fix with example corrected code.
- D** Run a second validation pass that filters out any findings without a suggested fix before returning results.

**Why C is correct:**

**Few-shot examples when instructions alone produce inconsistent output.** Detailed instructions grow linearly while edge cases grow combinatorially — prose instructions always leave room for interpretation. 2–4 concrete examples demonstrating the exact output shape (location → issue → why it

matters → specific fix → corrected code) give the model an output pattern to replicate consistently. Temperature (B) controls randomness, not output completeness. Filtering (D) hides the information without fixing it.



**Q9** ✗ INCORRECT

Your automated code review averages 15 findings per pull request, with developers reporting a 40% false positive rate. The bottleneck is investigation time: developers must click into each finding to read Claude's reasoning before deciding whether to address or dismiss it. Your CLAUDE.md already contains comprehensive rules for acceptable patterns, and stakeholders have rejected any approach that filters findings before developer review. What change would best address the investigation time bottleneck?

- A** Add a confidence score (1–10) to each finding so developers can sort by confidence and skip low-scoring items.
- B** Require Claude to include a brief inline rationale with each finding — the specific rule violated and why this instance isn't a known-acceptable pattern.
- C** Reduce the review scope to only security and correctness issues, eliminating the categories most prone to false positives.
- D** Run two independent review instances and surface only findings that appear in both.

**Why B is correct:**

**Inline rationale eliminates the click-through bottleneck without filtering.** Developers can read the specific rule violated and why it applies here directly in the findings list — no navigation required. This addresses investigation time while keeping all findings visible. Confidence scores (A) still require opening each finding to validate the reasoning. Scope reduction (C) and two-instance consensus (D) are forms of pre-review filtering — explicitly rejected by stakeholders.

## Q10 ✓ CORRECT



Your CI pipeline includes two Claude-powered code review modes: a pre-merge command that blocks PR merging until complete, and a 'deep analysis' mode that runs overnight, polls for batch completion, then posts detailed suggestions to the PR. You want to reduce API costs using the Message Batches API, which offers 50% cost savings but requires polling and may take up to 24 hours to complete. Which mode should use batch processing?

- A Both modes — the 50% savings applies equally and batch processing often completes in minutes.
- B The pre-merge hook only — it already has polling infrastructure and can absorb the latency.
- C The deep analysis only — it is latency-tolerant and already designed around async completion.
- D Neither — the Message Batches API doesn't support code review workflows.

**Why C is correct:**

**Match API to latency requirements.** The pre-merge hook blocks PR merging — someone is waiting on it. The Batches API has no guaranteed latency SLA and may take up to 24 hours: unsuitable for any blocking workflow. Deep analysis already runs overnight and polls for results — it is designed for async completion. Batch saves 50% and is a perfect fit for latency-tolerant non-blocking work.

## Q11 ✓ CORRECT

Your automated reviews identify valid issues but developers report the feedback isn't actionable. Findings say things like 'complex ticket allocation logic' or 'potential null pointer' without specifying what to change. When you add detailed instructions like 'always include specific fix suggestions,' the model still produces inconsistent output — sometimes detailed, sometimes vague. What prompting technique would most reliably produce consistently actionable feedback?

**A** Lower temperature to 0 to make output deterministic and consistently detailed



**B** Add more step-by-step instructions covering every possible finding type and how to suggest fixes for each.

**C** Add 2–4 few-shot examples showing complete, actionable findings — each demonstrating location, issue description, root cause, and a specific suggested fix with corrected code.

**D** Add a post-processing pass that filters out any findings without a suggested fix before returning results.

#### Why C is correct:

**Few-shot examples when detailed instructions still produce inconsistent output.** Prose instructions leave room for interpretation — edge cases grow combinatorially faster than instructions can cover them. 2–4 concrete examples of the exact output shape (location → issue → why it matters → specific fix → corrected code) give the model a pattern to replicate. Temperature (A) controls randomness, not output completeness or detail. More instructions (B) continues the failing approach. Filtering (D) hides the inconsistency without fixing it.

#### Q12 ✓ CORRECT

Your automated review analyzes comments and docstrings. The current prompt instructs Claude to 'check that comments are accurate and up-to-date.' Findings frequently flag acceptable patterns (TODO markers, straightforward descriptions) while missing comments that describe behavior the code no longer implements. What change addresses the root cause of this inconsistent analysis?

**A** Add few-shot examples showing correctly flagged and correctly ignored comment patterns.

**B** Replace the vague instruction with a specific categorical criterion: flag comments when the described behavior contradicts what the code actually does; skip TODO markers, style preferences, and description-only comments.

**C** Add a second review pass dedicated specifically to comment and docstring accuracy.

**D** Expand the review scope to include all inline comments, not just docstrings.

**Why B is correct:**

**'Check that comments are accurate' has no operational definition.** The model applies whatever judgment it was already applying — flagging what looks suspicious rather than what's actually wrong. The fix: replace the vague adjective with a categorical decision rule. 'Flag when described behavior contradicts actual code. Skip TODO markers, stylistic descriptions, straightforward summaries.' Few-shot examples (A) treat the symptom; a second pass (C) doubles cost without fixing the broken instruction.

**Q13** ✗ **INCORRECT**

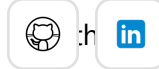
Your automated code review averages 15 findings per pull request with a 40% developer-reported false positive rate. The bottleneck is investigation time: developers must click into each finding to read Claude's reasoning before deciding whether to act or dismiss. Your CLAUDE.md contains comprehensive rules for acceptable patterns, and stakeholders have rejected any approach that filters findings before developer review. What single change would most reduce investigation time?

**A** Add a confidence score (1–10) to each finding so developers can triage by score without opening items.

**B** Require Claude to include a brief inline rationale with each finding — the specific rule violated and why this instance is not a known-acceptable pattern.

**C** Reduce the review scope to security and correctness issues only, removing the noisiest categories.

- D Run two independent review instances and only surface findings that appear



**Why B is correct:**

**Inline rationale eliminates the click-through without filtering.** Developers read rule + reasoning directly in the list — no navigation required. Confidence scores (A) still require opening the finding to validate whether the score is justified. Scope reduction (C) and consensus filtering (D) are pre-review filters — explicitly ruled out by stakeholders.

**Q14** ✗ INCORRECT

Your invoice extraction pipeline returns structured JSON but validation catches that extracted dates are formatted as MM/DD/YYYY when your schema requires ISO 8601 (YYYY-MM-DD). The rest of the extraction looks correct. What's the most effective fix?

- A Add 'return dates in ISO 8601 format' to the system prompt and re-run the full extraction.
- B Add a post-processing step that reformats all date strings using a regex before validation.
- C Send a follow-up request including the original invoice, the failed extraction output, and the specific validation error ('date field must be YYYY-MM-DD, got MM/DD/YYYY'), and ask the model to correct it.
- D Make the date field nullable and handle format normalisation in the application layer.

**Why C is correct:**

**Validation-retry with specific error feedback for format mismatches.** A wrong date format is an output error (wrong structure) — not absent data. This is exactly when retry-with-error-feedback works: the model self-corrects when given the original document, the failed output, and a specific error message. Cheaper than a full re-run. Nullable + app-layer (D) fixes fabrication, not format errors. Regex post-processing (B) works but fixes the symptom outside the model rather than correcting the

extraction.



**Next Section** →